

Mini PKI Project

Secure Document Signing and Verification System

CST8805 - Applied Cryptography -- Final Project

Algonquin College

Winter 2026

Date: April 12, 2026

Team Members

Mohammad Zghoul (zgho0001@algonquinlive.com)

Herman Yamdjeu Menadjou (yamd0001@algonquinlive.com)

Christianlan Tawambamanga (tawa0005@algonquinlive.com)

Omar Alzein (alze0011@algonquinlive.com)

1. System Overview

This project implements a mini Public Key Infrastructure (PKI) system that simulates secure document signing and verification. The system consists of three components:

- **Certification Authority (CA)** - Generates the root key pair and issues X.509 certificates to clients.
- **Signing Party (Client - Bob)** - Generates a key pair, obtains a certificate from the CA, and signs documents.
- **Relying Party (Verifier - Alice)** - Validates the certificate chain and verifies document signatures.

The system ensures document integrity, signer authentication, and non-repudiation using NIST-approved algorithms: RSA for key generation and digital signatures, and SHA-256 for hashing. All operations are performed using OpenSSL 3.5.5 on a single machine running Windows 11.

Algorithms and Tools Used

Component	Algorithm/Tool	Specification
Key Generation	RSA	4096-bit (CA), 2048-bit (Client)
Hashing	SHA-256 / SHA-512	NIST FIPS 180-4
Certificates	X.509 v3	ITU-T / RFC 5280
Tool	OpenSSL	Version 3.5.5
Automation	Python	Version 3.12 (subprocess)

2. Steps Performed

Step 1: CA Setup

A 4096-bit RSA private key was generated for the Certificate Authority. A self-signed X.509 v3 certificate was then created with SHA-256 signing, valid for 10 years. The certificate includes the CA's distinguished name attributes (CN, O, OU, L, ST, C, Email) and the Basic Constraints extension set to CA:TRUE, identifying it as a root certificate authority.

Commands Used:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out CA\ca_private.key
openssl req -x509 -new -key CA\ca_private.key -sha256 -days 3650 -out CA\ca_certificate.pem -subj
"/C=CA/ST=Ontario/L=Ottawa/O=MiniPKI Certificate Authority/OU=CA Department/CN=MiniPKI Root
CA/emailAddress=ca@minipki.local"
```

```
C:\Mini_PKI>openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out CA\ca_private.key
What it does: Creates a highly secure (4096-bit) private key using the standard RSA algorithm and saves it to a file named ca_private.key inside your CA folder.

C:\Mini_PKI>openssl pkey -in CA\ca_private.key -text -noout
Private-Key: (4096 bit, 2 primes)
modulus:
00:c3:31:49:e0:89:97:23:4d:c0:a9:98:86:50:43:
f4:c3:97:96:78:b1:09:5f:23:7f:12:d8:60:7a:3e:
ac:5a:da:36:de:f6:73:e9:f8:bf:be:da:cc:7a:a3:
ac:5b:61:fa:0b:7f:e0:e4:d5:ec:aa:58:2d:c3:ee:
6d:9c:0b:02:da:f1:4b:20:3b:cd:e2:fc:cd:cb:c4:
34:5f:db:3b:bd:03:17:56:12:49:7e:c0:a7:12:b3:
0a:4d:69:ec:c6:a9:cd:7b:ec:f6:53:d0:c7:c2:40:
91:be:0c:48:5b:05:30:1f:0e:41:a1:36:63:02:a6:
a8:9c:4c:22:1f:2f:71:6a:7b:a2:0c:26:fa:5a:59:
fc:cd:09:35:65:c5:c0:ab:0e:d2:db:95:7b:7b:ab:
75:a7:4f:0f:4f:a7:63:dd:3f:7e:c5:06:09:45:3f:
fa:43:a9:92:9e:7d:c7:8d:3c:24:d0:7e:04:12:cf:
3f:ac:9b:5d:3e:e4:8e:7d:d5:63:a3:e0:99:cc:a4:
c2:c8:aa:8d:e4:b7:e8:53:4a:32:f4:5f:f6:23:0f:
0e:91:fd:b2:86:b3:48:9e:a8:21:ab:ff:46:8a:6f:
27:49:26:bf:b7:c7:ea:d9:78:7f:e0:77:9d:5c:7a:
0e:e4:73:45:b2:87:e7:86:5e:9b:f7:93:ef:54:de:
c0:83:4c:0c:3b:89:62:fa:4a:c1:09:ac:fb:8c:0e:
30:a2:00:36:f6:2f:35:15:03:50:37:df:2d:f2:70:
5e:42:48:a7:7e:db:d2:25:aa:6e:fc:04:e7:ba:f1:
03:d5:30:2e:9e:4a:a9:62:68:0a:01:82:7e:39:67:
4b:e3:00:72:14:fe:9d:e8:0a:9a:f0:b8:da:de:73:
e5:06:63:75:12:21:27:b2:07:11:d9:af:0e:c8:ff:
4e:b6:a9:2a:c1:71:ec:17:3d:a0:7a:df:bc:33:03:
63:d8:ee:35:3b:7e:9c:1e:52:38:89:f4:27:b3:58:
6a:27:43:da:51:27:62:5a:0a:c2:90:9f:ce:b4:90:
9b:bc:5a:bf:64:05:0e:67:53:01:0a:07:fc:2f:54:
8d:a1:4e:49:b1:8c:78:ff:ea:45:b3:cc:12:b6:46:
49:61:fd:25:b7:f9:45:5e:e2:54:e8:2f:93:38:a8:
20:52:7c:e4:1f:0c:24:fc:d4:b6:b8:5a:02:7a:33:
3a:b7:11:b4:c1:63:f9:ca:17:7d:66:c4:d2:07:1d:
ab:13:c6:30:dd:c4:f2:b4:33:72:du:53:7f:c6:49:
60:b1:09:55:bf:a3:33:f5:8c:2e:2a:93:65:08:17:
17:5a:ff:d4:81:2c:c8:da:81:d5:99:78:1f:5d:17:
95:5a:23
```

[SUCCESS] CA private key (4096-bit RSA) generated successfully.

Figure 1: CA private key generation (RSA 4096-bit)

```
C:\Mini_PKI>openssl req -x509 -new -key CA\ca_private.key -sha256 -days 3650 -out CA\ca_certificate.pem -subj "/C=CA/ST=Ontario/L=Ottawa/O=MiniPKI Certificate Authority/OU=CA Department/CN=MiniPKI Root CA/emailAddress=ca@minipki.local"
This command Creates a self-signed X.509 certificate for the CA. Key flags: -x509 means create a certificate (not a CSR). -new generates a new request internally. -key specifies the private key to sign with. -sha256 uses the SHA-256 hash algorithm. -days 3650 sets validity to 10 years. The -subj flag sets the Distinguished Name (CN) with: C=Country, ST=State, L=Locality, O=Organization, OU=Organizational Unit, CN=Common Name, emailAddress. This certificate is self-signed because the CA is the root of trust. Note: The -subj value uses forward slashes as the DN separator (this is OpenSSL syntax, not a file path).
```

```
C:\Mini_PKI>openssl x509 -in CA\ca_certificate.pem -text -noout
Displays the full certificate in human-readable format, showing the version, serial number, signature algorithm, issuer, validity period, subject, public key info, and extensions. The Basic Constraints extension should show CA:TRUE, confirming this is a CA certificate.

Certificate:
Data:
  Version: 3 (0x2)
  Serial Number:
    70:4f:38:67:e2:2d:48:43:13:63:e4:bf:88:46:97:2f:11:a4:29:d9
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C = CA, ST = Ontario, L = Ottawa, O = MiniPKI Certificate Authority, OU = CA Department, CN = MiniPKI Root CA, emailAddress = ca@minipki.local
  Validity
    Not Before: Apr 13 01:18:52 2026 GMT
    Not After : Apr 10 01:18:52 2036 GMT
  Subject: C = CA, ST = Ontario, L = Ottawa, O = MiniPKI Certificate Authority, OU = CA Department, CN = MiniPKI Root CA, emailAddress = ca@minipki.local
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    Public-Key: (4096 bit)
    Modulus:
      00:c3:31:49:e0:89:97:23:4d:c0:a9:98:86:50:43:
      f4:c3:97:96:78:b1:09:5f:23:7f:12:d8:60:7a:3e:
      ac:5a:da:36:de:f6:73:e9:f8:bf:be:da:cc:7a:a3:
      ac:5b:61:fa:0b:7f:e0:e4:d5:ec:aa:58:2d:c3:ee:
      6d:9c:0b:02:da:f1:4b:20:3b:cd:e2:fc:cd:cb:c4:
      34:5f:db:3b:bd:03:17:56:12:49:7e:c0:a7:12:b3:
      0a:4d:69:ec:c6:a9:cd:7b:ec:f6:53:d0:c7:c2:40:
      91:be:0c:48:5b:05:30:1f:0e:41:a1:36:63:02:a6:
      a8:9c:4c:22:1f:2f:71:6a:7b:a2:0c:26:fa:5a:59:
      fc:cd:09:35:65:c5:c0:ab:0e:d2:db:95:7b:7b:ab:
      75:a7:4f:0f:4f:a7:63:dd:3f:7e:c5:06:09:45:3f:
      fa:43:a9:92:9e:7d:c7:8d:3c:24:d0:7e:04:12:cf:
      3f:ac:9b:5d:3e:e4:8e:7d:d5:63:a3:e0:99:cc:a4:
      c2:c8:aa:8d:e4:b7:e8:53:4a:32:f4:5f:f6:23:0f:
      0e:91:fd:b2:86:b3:48:9e:a8:21:ab:ff:46:8a:6f:
      27:49:26:bf:b7:c7:ea:d9:78:7f:e0:77:9d:5c:7a:
      0e:e4:73:45:b2:87:e7:86:5e:9b:f7:93:ef:54:de:
      c0:83:4c:0c:3b:89:62:fa:4a:c1:09:ac:fb:8c:0e:
      30:a2:00:36:f6:2f:35:15:03:50:37:df:2d:f2:70:
      5e:42:48:a7:7e:db:d2:25:aa:6e:fc:04:e7:ba:f1:
      03:d5:30:2e:9e:4a:a9:62:68:0a:01:82:7e:39:67:
      4b:e3:00:72:14:fe:9d:e8:0a:9a:f0:b8:da:de:73:
      e5:06:63:75:12:21:27:b2:07:11:d9:af:0e:c8:ff:
      4e:b6:a9:2a:c1:71:ec:17:3d:a0:7a:df:bc:33:03:
      63:d8:ee:35:3b:7e:9c:1e:52:38:89:f4:27:b3:58:
      6a:27:43:da:51:27:62:5a:0a:c2:90:9f:ce:b4:90:
      9b:bc:5a:bf:64:05:0e:67:53:01:0a:07:fc:2f:54:
      8d:a1:4e:49:b1:8c:78:ff:ea:45:b3:cc:12:b6:46:
      49:61:fd:25:b7:f9:45:5e:e2:54:e8:2f:93:38:a8:
      20:52:7c:e4:1f:0c:24:fc:d4:b6:b8:5a:02:7a:33:
      3a:b7:11:b4:c1:63:f9:ca:17:7d:66:c4:d2:07:1d:
      ab:13:c6:30:dd:c4:f2:b4:33:72:du:53:7f:c6:49:
      60:b1:09:55:bf:a3:33:f5:8c:2e:2a:93:65:08:17:
      17:5a:ff:d4:81:2c:c8:da:81:d5:99:78:1f:5d:17:
      95:5a:23
```

[SUCCESS] CA self-signed certificate generated. Valid for 10 years.

```
C:\Mini_PKI>openssl x509 -in CA\ca_certificate.pem -pubkey -noout > CA\ca_public.key
Extracts the public key from the CA certificate and saves it to a separate file. The -pubkey flag outputs the public key, and -noout suppresses the certificate output. This public key is what relying parties use to verify certificates issued by this CA.
```

Figure 2: CA self-signed certificate details

Explanation of Results:

The output confirms a 4096-bit RSA private key was generated with two prime factors. The self-signed certificate shows the Issuer and Subject are identical (self-signed), the signature algorithm is sha256WithRSAEncryption, and the Basic Constraints extension is set to CA:TRUE, which is required for a root CA. The certificate is valid from April 12, 2026 to April 9, 2036 (10 years).

Step 2: Client Key Generation and CSR

Bob generated a 2048-bit RSA key pair and extracted the public key. A Certificate Signing Request (CSR) was created containing Bob's identity information (CN=Bob Smith, O=Algonquin College, OU=CST Department) and his public key, signed with SHA-256.

Commands Used:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out Client\bob_private.key
```

```
openssl pkey -in Client\bob_private.key -pubout -out Client\bob_public.key
openssl req -new -key Client\bob_private.key -sha256 -out Client\bob_csr.pem -subj
"/C=CA/ST=Ontario/L=Ottawa/O=Algonquin College/OU=CST Department/CN=Bob
Smith/emailAddress=bob@algonquin.local"
```

```
C:\Mini_PKI>openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out Client\bob_private.key
.....
[SUCCESS] Client public/private key pair generated.
```

Figure 3: Client RSA 2048-bit key pair generation

```
C:\Mini_PKI>openssl req -new -key Client\bob_private.key -sha256 -out Client\bob_csr.pem -subj "/C=CA/ST=Ontario/L=Ottawa/O=Algonquin College/OU=CST Department/CN=Bob Smith/emailAddress=bob@algonquin.local"
.....
[SUCCESS] CSR generated and ready for submission to CA.
```

Figure 4: Certificate Signing Request (CSR) creation

Explanation of Results:

The public key was successfully extracted in PEM format. The CSR contains Bob's Distinguished Name (Subject: C=CA, ST=Ontario, L=Ottawa, O=Algonquin College, OU=CST Department, CN=Bob Smith) and his 2048-bit RSA public key. The CSR is signed with sha256WithRSAEncryption, proving Bob owns the corresponding private key. Importantly, the CSR does NOT contain the private key and is safe to transmit to the CA.

Step 3: Certificate Issuance

The CA signed Bob's CSR using its private key and SHA-256, producing an X.509 certificate valid for 365 days. The issued certificate links Bob's identity to his public key, signed by the trusted CA. The certificate was verified against the CA certificate to confirm the trust chain.

Commands Used:

```
openssl x509 -req -in Client\bob_csr.pem -CA CA\ca_certificate.pem -CAkey CA\ca_private.key
-Acreateserial -sha256 -days 365 -out Client\bob_certificate.pem
openssl verify -CAfile CA\ca_certificate.pem Client\bob_certificate.pem
```

```
C:\Mini_PKI>openssl x509 -req -in Client\bob_csr.pem -CA CA\ca_certificate.pem -CAkey CA\ca_private.key -Acreateserial -sha256 -days 365 -out Client\bob_certificate.pem
Certificate request self-signature ok
subject=C = CA, ST = Ontario, L = Ottawa, O = Algonquin College, OU = CST Department, CN = Bob Smith, emailAddress = bob@algonquin.local

C:\Mini_PKI>openssl x509 -in Client\bob_certificate.pem -text -noout
Displays the issued certificate, showing that the Issuer is the CA (MiniPKI Root CA) and the Subject is Bob Smith. The certificate contains Bob's public key and is signed by the CA's private key.
Certificate:
  Data:
    Version: 1 (0x0)
    Serial Number:
      65:17:83:7b:bc:fa:db:0c:66:6a:61:d0:e8:e5:51:f9:10:3d:c1:08
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = CA, ST = Ontario, L = Ottawa, O = MiniPKI Certificate Authority, OU = CA Department, CN = MiniPKI Root CA, emailAddress = ca@algonquin.local
    Validity
      Not Before: Apr 13 01:50:33 2026 GMT
      Not After : Apr 13 01:50:33 2027 GMT
    Subject: C = CA, ST = Ontario, L = Ottawa, O = Algonquin College, OU = CST Department, CN = Bob Smith, emailAddress = bob@algonquin.local
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:a4:cb:f9:a9:0d:b0:ae:8d:74:6d:58:7d:b0:1d:
        a6:04:f3:fd:77:92:3c:91:b7:ab:65:38:52:3e:2f:
        0e:9d:c6:8e:f8:a8:07:99:d2:6f:9e:4e:bf:8d:22:
        1d:29:5c:af:cb:cb:15:1c:f2:96:ca:d7:62:7c:ce:
        c2:a1:57:d5:b6:4d:e7:b9:d6:61:be:cf:d1:a0:b5:
        16:69:09:a3:ef:f6:d6:8f:16:d9:36:03:ab:fa:79:
        69:4a:a2:96:e3:bc:a3:1f:5a:1c:05:ab:6f:a8:b0:
        07:77:45:e7:db:46:52:34:26:ba:f2:69:2b:3a:b3:
        4e:bb:04:c1:bd:a3:42:8f:21:71:98:bc:72:52:bb:
        8b:78:58:45:73:48:70:dd:24:4f:b4:64:a2:15:
        bf:ee:1d:cd:c6:33:16:e4:fd:4f:b6:cd:f9:26:b0:
        97:b3:fa:33:fc:d1:12:56:c1:32:e0:de:32:14:46:
        c6:aa:4b:36:fb:c2:0c:22:32:d5:4b:f6:34:72:04:
        e8:4e:ca:2a:d3:34:09:21:18:4c:e8:8f:e9:2c:23:
        25:7d:04:e6:b3:61:c6:fd:9d:f9:b4:63:04:76:65:
        ae:ff:de:ea:01:9e:1e:46:eb:2a:20:bc:52:3b:cc:
        d8:4d:5a:82:70:7b:a7:bc:c6:dd:5f:e6:5b:93:3d:
        ee:27
      Public-Key: (2048 bit)
      Modulus:
        00:a4:cb:f9:a9:0d:b0:ae:8d:74:6d:58:7d:b0:1d:
        a6:04:f3:fd:77:92:3c:91:b7:ab:65:38:52:3e:2f:
        0e:9d:c6:8e:f8:a8:07:99:d2:6f:9e:4e:bf:8d:22:
        1d:29:5c:af:cb:cb:15:1c:f2:96:ca:d7:62:7c:ce:
        c2:a1:57:d5:b6:4d:e7:b9:d6:61:be:cf:d1:a0:b5:
        16:69:09:a3:ef:f6:d6:8f:16:d9:36:03:ab:fa:79:
        69:4a:a2:96:e3:bc:a3:1f:5a:1c:05:ab:6f:a8:b0:
        07:77:45:e7:db:46:52:34:26:ba:f2:69:2b:3a:b3:
        4e:bb:04:c1:bd:a3:42:8f:21:71:98:bc:72:52:bb:
        8b:78:58:45:73:48:70:dd:24:4f:b4:64:a2:15:
        bf:ee:1d:cd:c6:33:16:e4:fd:4f:b6:cd:f9:26:b0:
        97:b3:fa:33:fc:d1:12:56:c1:32:e0:de:32:14:46:
        c6:aa:4b:36:fb:c2:0c:22:32:d5:4b:f6:34:72:04:
        e8:4e:ca:2a:d3:34:09:21:18:4c:e8:8f:e9:2c:23:
        25:7d:04:e6:b3:61:c6:fd:9d:f9:b4:63:04:76:65:
        ae:ff:de:ea:01:9e:1e:46:eb:2a:20:bc:52:3b:cc:
        d8:4d:5a:82:70:7b:a7:bc:c6:dd:5f:e6:5b:93:3d:
        ee:27
      Exponent: 65537 (0x10001)
  Verifies that Bob's certificate was legitimately issued by the trusted CA. The -CAfile flag specifies the trusted CA certificate. If valid, OpenSSL outputs "OK". This is the same verification a relying party performs to establish trust.

C:\Mini_PKI>openssl verify -CAfile CA\ca_certificate.pem Client\bob_certificate.pem
Client\bob_certificate.pem: OK
```

Figure 5: CA issues X.509 certificate for Bob and verification against CA

Explanation of Results:

The CA successfully signed Bob's CSR and issued an X.509 v3 certificate. The output shows the Issuer is the CA (MiniPKI Root CA) and the Subject is Bob Smith. The certificate serial number uniquely identifies this certificate. The Authority Key Identifier matches the CA's Subject Key Identifier, confirming the chain of trust. The final verification command outputs "OK", proving the certificate is legitimately signed by the trusted CA.

Step 4: Document Signing

Bob created a plain-text document and signed it using his private key with SHA-256. The signing process computes a SHA-256 hash of the document, then encrypts the hash with Bob's RSA private key. The result is stored as signature.bin.

Commands Used:

```
echo "This is a confidential document signed by Bob Smith." > Client\document.txt
openssl dgst -sha256 -sign Client\bob_private.key -out Client\signature.bin Client\document.txt
openssl dgst -sha256 Client\document.txt
```

```
C:\Mini_PKI>echo "This is a confidential document signed by Bob Smith. Project: Mini PKI System for CST8805 Applied Cryptography. This document verifies that the PKI system is fully operational." > Client\document.txt

C:\Mini_PKI>openssl dgst -sha256 -sign Client\bob_private.key -out Client\signature.bin Client\document.txt
This is the core signing command. It performs two operations: (1) computes a SHA-256 hash of document.txt, and (2) encrypts (signs) that hash with Bob's RSA private key. The result is saved as signature.bin. The dgst command with -sign performs a digital signature operation. Anyone with Bob's public key can verify this signature, but only Bob could have created it.
Creates a plain-text document that Bob will sign. In a real-world scenario, this could be any file (PDF, contract, code, etc.).

C:\Mini_PKI>openssl dgst -sha256 Client\document.txt
Computes and displays the SHA-256 hash of the document.
SHA2-256(Client\document.txt)= 74a13221b7e7444e752f398fed2ab2d90b4e0c9f9f701628522b89e7486c28b3

C:\Mini_PKI>openssl base64 -in Client\signature.bin
Displays the binary signature file in Base64 encoding for human-readable use. The actual signature bin is a binary file. Base64 encodes it as text.
c1oVcK6dPjA1I7Mk0pna0b0H4Ltm8T8spLx4B0jgYp2p0b
Y2uFe+3zPqzB8R0n+5MGLyZdL+t1Sk4Fpcub3BqMREZ5c2tA1QAAZw0p
sKRZ3j+QueKab1FTN6s6QCmLjD8VQGuExj1WQchRU+yxBQ/OczU78oP9JqV
Nm9Sqb72GnTEB3qbxJVmB4vbrpTpdNjDULN6G0upkPp23wqIDmHPRkzLS0z7
rRwXCKL1A4s76cJcZwtmQ1BCQZDoomXDVgyu0PKswJhdgLdocdEX6sJhGwL
xq3LMSK60ecZfV1D9u==

[SUCCESS] Document signed. Signature stored in Client/signature.bin

C:\Mini_PKI>
```

Figure 6: Document creation, signing, and hash computation

Explanation of Results:

The document was signed successfully, producing a 256-byte binary signature file (signature.bin). The SHA-256 hash of the document is displayed as a 64-character hexadecimal string (256 bits). This

signature provides three security guarantees: integrity (any change to the document invalidates the signature), authentication (only Bob's private key could have created it), and non-repudiation (Bob cannot deny signing since only he possesses the private key).

3. Verification Results

Valid Case - Successful Verification

Alice (the verifier) first verified Bob's certificate against the trusted CA certificate, confirming the chain of trust. She then extracted Bob's public key from his certificate and used it to verify the document signature.

Commands Used:

```
openssl verify -CAfile CA\ca_certificate.pem Client\bob_certificate.pem
openssl x509 -in Client\bob_certificate.pem -pubkey -noout > Verifier\bob_pubkey_from_cert.pem
openssl dgst -sha256 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature.bin Client\document.txt
```

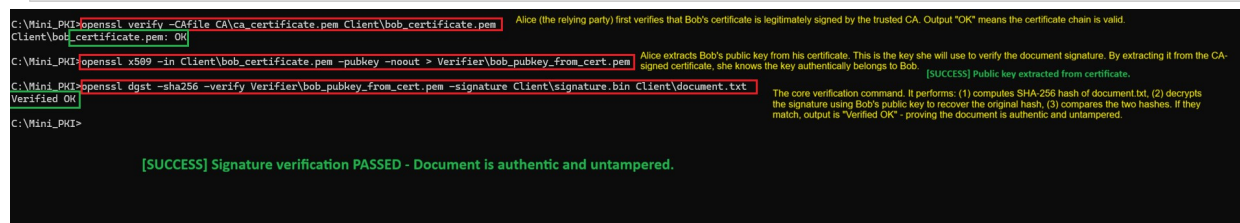


Figure 7: Successful signature verification (Valid Case)

Explanation of Results:

OpenSSL outputs "Verified OK", confirming the document is authentic and has not been tampered with. The verification process: (1) computed the SHA-256 hash of the received document, (2) decrypted the signature using Bob's public key to recover the original hash, (3) compared the two hashes and found them identical. This proves the document was signed by Bob and has not been modified since signing.

Invalid Cases - Failed Verification

Two invalid scenarios were tested to demonstrate the system correctly detects both document tampering and authentication fraud.

Commands Used:

Test 1 - Tampered Document:

```
copy Client\document.txt Verifier\tampered_document.txt
echo "TAMPERED: This line was added by an attacker." >> Verifier\tampered_document.txt
openssl dgst -sha256 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature.bin Verifier\tampered_document.txt
```

Test 2 - Wrong Key (Attacker):

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out Verifier\attacker_private.key
openssl pkey -in Verifier\attacker_private.key -pubout -out Verifier\attacker_public.key
openssl dgst -sha256 -verify Verifier\attacker_public.key -signature Client\signature.bin Client\document.txt
```



```

C:\Mini_PKI>copy Client\document.txt Verifier\tampered_document.txt
1 file(s) copied.
Creates a copy of the original document using the Windows copy command, then appends an extra line to simulate tampering. The tampered document has a different SHA-256 hash than the original.

C:\Mini_PKI>echo "TAMPERED: This line was added by an attacker." >> Verifier\tampered_document.txt

C:\Mini_PKI>openssl dgst -sha256 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature.bin Verifier\tampered_document.txt
Verification failure
09760800:error:02000084:rsa routines:ossl_rsa_verify:bad signature:crypto\rsa\rsa_sign.c:439:
40760800:error:1C800084:Provider routines:rsa_verify:RSA lib:providers\implementations\signature\rsa_sig.c:774:
Attempts to verify the original signature against the tampered document. Output: "Verification failure". This fails because the SHA-256 hash of the tampered document does not match the hash embedded in the signature. This demonstrates document integrity - any modification is detected.

C:\Mini_PKI>openssl genkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out Verifier\attacker_private.key
Creates a separate RSA key pair for an attacker. This simulates someone trying to impersonate Bob.

C:\Mini_PKI>openssl pkey -in Verifier\attacker_private.key -pubout -out Verifier\attacker_public.key

C:\Mini_PKI>openssl dgst -sha256 -verify Verifier\attacker_public.key -signature Client\signature.bin Client\document.txt
Verification failure
09760800:error:02000084:rsa routines:RSA_padding_check_PKCS1_type_1:invalid padding:crypto\rsa\rsa_pk1.c:75:
040760800:error:02000072:rsa routines:rsa_ossl_public_decrypt:padding check failed:crypto\rsa\rsa_ossl.c:598:
040760800:error:1C800084:Provider routines:rsa_verify:RSA lib:providers\implementations\signature\rsa_sig.c:774:
Attempts to verify Bob's signature using the attacker's public key. Output: "Verification failure". This fails because the signature was created with Bob's private key, and only Bob's corresponding public key can decrypt it. This demonstrates authentication - the signature proves the signer's identity.

C:\Mini_PKI>
Verification failure
Reason: Signature was not created with this key - authentication failed.

```

Figure 8: Failed verification - tampered document and wrong key

Explanation of Results:

Both invalid tests correctly output "Verification failure". In the tampered document test, the SHA-256 hash of the modified document does not match the hash stored in the signature, proving document integrity is compromised. In the wrong key test, the attacker's public key cannot decrypt a signature created with Bob's private key, proving authentication failure. These results demonstrate the PKI system reliably detects both types of attacks.

4. Optional Features

4.1 Hash Algorithm Comparison (SHA-256 vs SHA-512)

The document was signed with both SHA-256 and SHA-512 to compare the two NIST-approved hash algorithms.

Commands Used:

```

openssl dgst -sha256 Client\document.txt
openssl dgst -sha512 Client\document.txt
openssl dgst -sha512 -sign Client\bob_private.key -out Client\signature_sha512.bin Client\document.txt
openssl dgst -sha512 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature_sha512.bin Client\document.txt
openssl dgst -sha256 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature_sha512.bin Client\document.txt

```

```

C:\Mini_PKI>openssl dgst -sha512 -sign Client\bob_private.key -out Client\signature_sha512.bin Client\document.txt
Signs the same document using SHA-512 instead of SHA-256. SHA-512 produces a 512-bit hash (128 hex characters) compared to SHA-256's 256-bit hash (64 hex characters). However, the signature file size remains the same (256 bytes) because it is determined by the RSA key size (2048 bits = 256 bytes), not the hash size.

C:\Mini_PKI>openssl dgst -sha512 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature_sha512.bin Client\document.txt
Verified OK
Verifies the SHA-512 signature. Output: "Verified OK". The verifier must use the same hash algorithm (SHA-512) that the signer used.

C:\Mini_PKI>openssl dgst -sha256 -verify Verifier\bob_pubkey_from_cert.pem -signature Client\signature_sha512.bin Client\document.txt
Verification failure
88960800:error:02000084:rsa routines:ossl_rsa_verify:bad signature:crypto\rsa\rsa_sign.c:439:
88960800:error:1C800084:Provider routines:rsa_verify:RSA lib:providers\implementations\signature\rsa_sig.c:774:
Attempts to verify a SHA-512 signature using SHA-256. Output: "Verification failure". This proves that the verifier must know and use the same hash algorithm as the signer. In practice, the algorithm is typically encoded in the signature or specified in the certificate.

C:\Mini_PKI>

```

Figure 9: SHA-256 vs SHA-512 hash comparison and cross-algorithm test

Explanation of Results:

SHA-256 produces a 64-character hex hash (256 bits), while SHA-512 produces a 128-character hex hash (512 bits). Both signature files are 256 bytes because the RSA-2048 key determines the signature size, not the hash algorithm. SHA-512 verification succeeded ("Verified OK"), but cross-algorithm verification (verifying a SHA-512 signature with SHA-256) correctly failed with "Verification failure". This proves the verifier must use the same hash algorithm as the signer.

4.2 Certificate Expiration Simulation

An expired certificate was created to demonstrate that the PKI system enforces certificate validity periods.

Commands Used:

```
openssl x509 -req -in Client\bob_csr.pem -CA CA\ca_certificate.pem -CAkey CA\ca_private.key
-CAcreateserial -sha256 -not_before 20250101000000Z -not_after 20250201000000Z -out
Verifier\bob_expired_cert.pem
openssl x509 -in Verifier\bob_expired_cert.pem -noout -dates
openssl verify -CAfile CA\ca_certificate.pem Verifier\bob_expired_cert.pem
```

```
C:\Mini_PKI>openssl x509 -req -in Client\bob_csr.pem -CA CA\ca_certificate.pem -CAkey CA\ca_private.key -CAcreateserial -sha256 -not_before 20250101000000Z -not_after 20250201000000Z -out Verifier\bob_expired_cert.pem
Certificate request self-signature ok
subject=C=CA, ST=Ontario, L=Ottawa, O=Algonquin College, OU=CST Department, CN=Bob Smith, emailAddress=bob@algonquin.local
C:\Mini_PKI>openssl x509 -in Verifier\bob_expired_cert.pem -noout -dates
notBefore=Jan  1 00:00:00 2025 GMT
notAfter=Feb  1 00:00:00 2025 GMT
C:\Mini_PKI>openssl verify -CAfile CA\ca_certificate.pem Verifier\bob_expired_cert.pem
error 10 at 0 depth lookup: certificate has expired
error Verifier\bob_expired_cert.pem: verification failed
C:\Mini_PKI>
```

Figure 10: Expired certificate verification (correctly rejected)

Explanation of Results:

The certificate was issued with validity from January 1, 2025 to February 1, 2025, which has already passed. When OpenSSL attempts to verify this certificate, it returns "error 10 at 0 depth lookup: certificate has expired", correctly rejecting it. This demonstrates that PKI systems enforce certificate lifecycle management. In real-world scenarios, expired certificates must be renewed to prevent the use of potentially compromised credentials.

4.3 Certificate Revocation (Manual List)

A manual certificate revocation list (CRL) was implemented to simulate the revocation of a compromised certificate. Bob's certificate serial number was extracted and added to a text-based revocation list. A verification check compares the certificate's serial against the revocation list before proceeding with signature verification.

Commands Used:

```
openssl x509 -in Client\bob_certificate.pem -noout -serial
openssl x509 -in Verifier\bob_expired_cert.pem -noout -serial
openssl x509 -in Client\bob_certificate.pem -noout -serial > CA\revoked_certs.txt
type CA\revoked_certs.txt
for /f "tokens=2 delims==" %A in ('openssl x509 -in Client\bob_certificate.pem -noout -serial') do @set BOB=%A
findstr /i "%BOB%" CA\revoked_certs.txt >nul && echo Bob's cert (%BOB%): REVOKED - signature verification should be DENIED || echo Bob's cert (%BOB%): VALID - signature verification can proceed
for /f "tokens=2 delims==" %A in ('openssl x509 -in Verifier\bob_expired_cert.pem -noout -serial') do @set OLD=%A
findstr /i "%OLD%" CA\revoked_certs.txt >nul && echo Older cert (%OLD%): REVOKED - signature verification should be DENIED || echo Older cert (%OLD%): NOT in CRL - CRL check passes
```



```

C:\Mini_PKI>openssl x509 -in Client\bob_certificate.pem -noout -serial
serial=6517837B8CFAD8C666A61D8E8E551F9103DC108
"Extract Bob's current cert serial number from its X.509 data."
"Bob's unique certificate serial -- this is what will be 'revoked'."

C:\Mini_PKI>openssl x509 -in Verifier\bob_expired_cert.pem -noout -serial
serial=6517837B8CFAD8C666A61D8E8E551F9103DC109
"Extract the expired cert's serial (different value) to use as the non-revoked control."
"Different serial -- proves a successful CRL lookup truly depends on a match."

C:\Mini_PKI>openssl x509 -in Client\bob_certificate.pem -noout -serial > CA\revoked_certs.txt
"Write Bob's serial line into the manual revocation list file."

C:\Mini_PKI>type CA\revoked_certs.txt
serial=6517837B8CFAD8C666A61D8E8E551F9103DC108
"Confirm the CRL now contains exactly Bob's serial."
"CRL content verified -- ready to check certificates against it."

C:\Mini_PKI>for /f "tokens=2 delims==" %A in ('openssl x509 -in Client\bob_certificate.pem -noout -serial') do @set BOB=%A
"Parse the OpenSSL output and store Bob's serial in the variable %BOB%."

C:\Mini_PKI>findstr /i "%BOB%" CA\revoked_certs.txt >nul && echo Bob's cert (%BOB%): REVOKED - signature verification should be DENIED || echo Bob's cert (%BOB%): VALID - signature verification can proceed
Bob's cert (6517837B8CFAD8C666A61D8E8E551F9103DC108): REVOKED - signature verification should be DENIED
"Match found -> CRL correctly blocks Bob's compromised certificate."

C:\Mini_PKI>for /f "tokens=2 delims==" %A in ('openssl x509 -in Verifier\bob_expired_cert.pem -noout -serial') do @set OLD=%A
"Store the second (different) serial in %OLD%."

C:\Mini_PKI>findstr /i "%OLD%" CA\revoked_certs.txt >nul && echo Older cert (%OLD%): REVOKED - signature verification should be DENIED || echo Older cert (%OLD%): NOT in CRL - CRL check passes
Older cert (6517837B8CFAD8C666A61D8E8E551F9103DC109): NOT in CRL - CRL check passes
"No match -> CRL correctly lets a non-revoked serial through, confirming the check isn't flagging everything."
"Same CRL lookup logic, applied to a different serial."

C:\Mini_PKI>

```

Figure 11: Certificate revocation check using manual CRL

Explanation of Results:

Bob's currently issued certificate serial was extracted with OpenSSL and written into a plain-text revocation list (CA\revoked_certs.txt). A cmd FOR loop captured the serial into a variable, and findstr was used to look it up inside the CRL. When the serial is found, findstr exits with code 0, triggering the && branch that prints "REVOKED - signature verification should be DENIED"; when absent, findstr exits with code 1, triggering the || branch that prints "VALID - signature verification can proceed". To confirm the check is not simply flagging everything, a second certificate with a different serial (the expired test certificate from Section 4.2) was checked against the same CRL and was correctly reported as "NOT in CRL - CRL check passes". This simulates the real-world CRL mechanism where CAs publish lists of revoked certificate serial numbers that relying parties must consult before trusting a certificate.

4.4 Automated PKI Process (Python Script)

A Python automation script (pki_automation.py) was developed to automate the entire PKI workflow end-to-end. The script uses Python's subprocess module to invoke OpenSSL commands programmatically. It automates all six phases: CA setup, client key generation, certificate issuance, document signing, signature verification (valid + invalid cases), and certificate revocation checking.

Commands Used:

```
python pki_automation.py
```

```

C:\Mini_PKI_Automation\python_pki_automation.py
=====
KALI: PKI AUTOMATION SCRIPT
CSTR005 - Applied Cryptography | Algoquote Collaps

[OK] openssl: found: C:\Program Files\OpenSSL\bin\openssl.exe
[OK] Directory structure created.

=====
PHASE 1: CERTIFICATION AUTHORITY (CA) SETUP
=====

-- Step 1a: Generate CA Private Key (RSA 4096-bit) --
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out C:\Mini_PKI_Automation\python_pki_output\CA\ca_private.key
[OK] CA private key generated. "4096-bit key produced - stronger than client key (2048) because the CA signs every cert."

-- Step 1b: Create CA Self-Signed Certificate (360=36*10 years) --
$ openssl req -x509 -new -x509s -key C:\Mini_PKI_Automation\python_pki_output\CA\ca_private.key -sha256 -days 3650 -out C:\Mini_PKI_Automation\python_pki_output\CA\ca_certificate.pem -subj /C=CA/ST=Ontario/L=Ottawa/O=MiniPKI Certificate Authority/OU=CA Department/CN=MiniPKI Root CA/emailAddress=cabin@ipis.local
[OK] CA self-signed certificate created. "Root of trust is now established."

-- Step 1c: Verify CA Certificate --
$ openssl verify -CA C:\Mini_PKI_Automation\python_pki_output\CA\ca_certificate.pem -noout -subj -issuer -dates
subject=CA, ST=Ontario, L=Ottawa, O=MiniPKI Certificate Authority, OU=CA Department, CN=MiniPKI Root CA, emailAddress=cabin@ipis.local
issuer=CA, ST=Ontario, L=Ottawa, O=MiniPKI Certificate Authority, OU=CA Department, CN=MiniPKI Root CA, emailAddress=cabin@ipis.local
notAfter=Apr 10 17:42:06 2036 GMT
[OK] CA certificate verified. "Subject and Issuer are identical -> confirms this is a self-signed root certificate."
"10 year validity window, as required by the spec."

=====
PHASE 2: CLIENT (BOB) KEY GENERATION & CSR
=====

-- Step 2a: Generate Client Private Key (RSA 2048-bit) --
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out C:\Mini_PKI_Automation\python_pki_output\client\bob_private.key
[OK] Client private key generated. "Generate Bob's 2048-bit RSA private key. Smaller than CA's because clients re-key more often."

-- Step 2b: Extract Client Public Key --
$ openssl pkcs8 -topk8 -inform PEM -outform DER -in C:\Mini_PKI_Automation\python_pki_output\client\bob_private.key -pubout -out C:\Mini_PKI_Automation\python_pki_output\client\bob_public.key
[OK] Client public key extracted. "Extract Bob's public key from his private key - safe to share."

-- Step 2c: Generate Certificate Signing Request (CSR) --
$ openssl req -new -key C:\Mini_PKI_Automation\python_pki_output\client\bob_private.key -sha256 -out C:\Mini_PKI_Automation\python_pki_output\client\bob_csr.pem -subj /C=CA/ST=Ontario/L=Ottawa/O=Algoquote Collaps/OU=CA Department/CN=Bob Smith/emailAddress=bob@algonquin.local
[OK] CSR generated. "Build a Certificate Signing Request containing Bob's identity and public key, signed with his private key."

-- Step 2d: Verify CSR Contents --
$ openssl req -text -noout -key C:\Mini_PKI_Automation\python_pki_output\client\bob_csr.pem -noout -subject
subject=CA, ST=Ontario, L=Ottawa, O=Algoquote Collaps, OU=CA Department, CN=Bob Smith, emailAddress=bob@algonquin.local
[OK] CSR contents verified. "CSR holds Bob's Distinguished Name + public key. Note: NO private key inside - safe to send to the CA."

=====
PHASE 3: CERTIFICATE ISSUANCE
=====

-- Step 3a: CA Signs CSR and Issues Certificate (365 days) --
$ openssl x509 -req -in C:\Mini_PKI_Automation\python_pki_output\client\bob_csr.pem -CA C:\Mini_PKI_Automation\python_pki_output\CA\ca_certificate.pem -CAkey C:\Mini_PKI_Automation\python_pki_output\CA\ca_private.key -CAcreateserial -sha256 -days 365 -out C:\Mini_PKI_Automation\python_pki_output\client\bob_certificate.pem
[OK] Certificate issued. "CA signs Bob's CSR using its private key, producing an X.509 v3 certificate valid for 365 days."

-- Step 3b: Verify Certificate Against CA --
$ openssl verify -CA C:\Mini_PKI_Automation\python_pki_output\CA\ca_certificate.pem C:\Mini_PKI_Automation\python_pki_output\client\bob_certificate.pem
[OK] Certificate verified. "Trust chain verified -> Bob's cert is legitimately signed by the MiniPKI Root CA."

=====
PHASE 4: DOCUMENT SIGNING
=====

-- Step 4a: Create Document --
[OK] Document created. C:\Mini_PKI_Automation\python_pki_output\client\document.txt
"Plain text document created programmatically (4 lines describing the project)."

-- Step 4b: Sign Document with SHA-256 --
$ openssl dgst -sha256 -sign C:\Mini_PKI_Automation\python_pki_output\client\bob_private.key -out C:\Mini_PKI_Automation\python_pki_output\client\signature.bin C:\Mini_PKI_Automation\python_pki_output\client\document.txt
[OK] Document signed. Signature saved to signature.bin
"256-byte binary signature produced. Guarantees integrity, authentication, and non-repudiation."
"Sign the document: SHA-256 hash -> encrypted with Bob's private key -> written to signature.bin."

-- Step 4c: Compute Document Hash --
$ openssl dgst -sha256 -r C:\Mini_PKI_Automation\python_pki_output\client\document.txt
[OK] Document hash computed. "64-character (256-bit) hash fingerprint of the document - any change to the file will produce a completely different hash."

=====
PHASE 5: SIGNATURE VERIFICATION
=====

-- Step 5a: Extract Public Key from Certificate --
$ openssl x509 -in C:\Mini_PKI_Automation\python_pki_output\client\bob_certificate.pem -pubkey -noout -out C:\Mini_PKI_Automation\python_pki_output\Verifier\bob_pubkey_from_cert.pem
[OK] Public key extracted from certificate. "Extract Bob's public key from his certificate and save it to Verifier\bob_pubkey_from_cert.pem. This is the exact key the verifier uses."

-- Step 5b: VALID CASE - Verify Original Document --
$ openssl dgst -sha256 -verify C:\Mini_PKI_Automation\python_pki_output\Verifier\bob_pubkey_from_cert.pem -signature C:\Mini_PKI_Automation\python_pki_output\client\signature.bin C:\Mini_PKI_Automation\python_pki_output\client\document.txt
[OK] Document verified. Signature matches. "VALID case: verify the original signature against the original document using Bob's public key."
"Positive test passed - PKI correctly accepts a legitimate signature."

-- Step 5c: INVALID CASE 1 - Tampered Document --
$ openssl dgst -sha256 -verify C:\Mini_PKI_Automation\python_pki_output\Verifier\bob_pubkey_from_cert.pem -signature C:\Mini_PKI_Automation\python_pki_output\client\signature.bin C:\Mini_PKI_Automation\python_pki_output\Verifier\tampered_document.txt
[OK] Document verification FAILED. "INVALID case 1: take a copy of the document, append an attacker line, then try to verify with the original signature."
"Tampered document correctly REJECTED."
"Recomputed hash doesn't match the one inside the signature -> tampering detected."

-- Step 5d: INVALID CASE 2 - Wrong Key (Attacker) --
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out C:\Mini_PKI_Automation\python_pki_output\Verifier\attacker_private.key -noout
$ openssl pkcs8 -topk8 -inform PEM -outform DER -in C:\Mini_PKI_Automation\python_pki_output\Verifier\attacker_private.key -pubout -out C:\Mini_PKI_Automation\python_pki_output\Verifier\attacker_public.key
$ openssl dgst -sha256 -verify C:\Mini_PKI_Automation\python_pki_output\client\document.txt -signature C:\Mini_PKI_Automation\python_pki_output\client\signature.bin -pubkey C:\Mini_PKI_Automation\python_pki_output\Verifier\attacker_public.key
[OK] Document verification FAILED. "INVALID case 2: attacker generates a completely different 2048-bit key pair and tries to validate Bob's signature with their own public key."
"Verification failure"
"Attacker's public key can't decrypt a signature made with Bob's private key -> RSA padding fails on the garbled output."
"Authentication check working as expected - signatures cannot be validated by impersonators."

=====
PHASE 6: CERTIFICATE REVOCATION CHECK
=====

-- Step 6a: Extract Certificate Serial Number --
$ openssl x509 -in C:\Mini_PKI_Automation\python_pki_output\client\bob_certificate.pem -noout -serial
[OK] Serial number extracted. "Extract Bob's certificate serial number for the CRL lookup."
Bob's serial: 2BE2952A1E37829B791504E791CB93113411EFA5
"Unique per-certificate identifier assigned by the CA. This is what the CRL will find."

-- Step 6b: Create Revocation List --
$ openssl ca -gencrl -config C:\Mini_PKI_Automation\python_pki_output\CA\revoked_certs.txt -serial 2BE2952A1E37829B791504E791CB93113411EFA5
[OK] Revocation list saved to C:\Mini_PKI_Automation\python_pki_output\CA\revoked_certs.txt
"Write Bob's serial into CA\revoked_certs.txt (the manual CRL)."
"Contains exactly one entry - Bob's compromised certificate."
"Script re-reads the CRL and checks whether Bob's serial is present."
"Match found -> revocation logic triggers."

-- Step 6c: Check Certificate Against Revocation List --
$ openssl ca -checkcrl -config C:\Mini_PKI_Automation\python_pki_output\CA\revoked_certs.txt -serial 2BE2952A1E37829B791504E791CB93113411EFA5
[OK] Certificate is REVOKED. "Match found -> revocation logic triggers."

=====
ALL PHASES COMPLETED SUCCESSFULLY
=====

Summary of files generated:
CA\ca_certificate.pem
CA\ca_certificate.srl
CA\ca_private.key
CA\revoked_certs.txt
Client\bob_certificate.pem
Client\bob_csr.pem
Client\bob_private.key
Client\bob_public.key
Client\document.txt
Client\signature.bin
Verifier\attacker_private.key
Verifier\attacker_public.key
Verifier\bob_pubkey_from_cert.pem
Verifier\tampered_document.txt

[NOTE] PKI automation complete.

```

Figure 12a & 12b: Python automation script executing the complete PKI workflow

Explanation of Results:

For readability, the console run is split across two figures. Figure 12a covers Phases 1–4: the script first confirmed the OpenSSL binary, auto-created the CA/Client/Verifier subfolders inside `python_pki_output\`, generated the CA's 4096-bit RSA private key and self-signed X.509 certificate (10-year validity, Subject = Issuer confirming the root), then generated Bob's 2048-bit key pair and CSR, had the CA sign the CSR to issue Bob's certificate (365 days, trust chain verified with "OK"), and finally created and signed a plain-text document with SHA-256, producing `signature.bin` and a 64-character hex hash fingerprint. Figure 12b covers Phases 5–6 and the completion summary: Bob's public key was extracted from his certificate; the valid case returned "Verified OK" and [PASS] Signature verification SUCCEEDED; the tampered-document case failed with an RSA "bad signature" error and was correctly [PASS] REJECTED; the wrong-key case failed with an RSA "padding check failed" error and was correctly [PASS] REJECTED; the revocation phase extracted Bob's serial (2BE2952A1E37829B7915D4E791CB93113411EFA5), wrote it to `CA\revoked_certs.txt`, and the CRL lookup returned [REVOKED] – Verification DENIED. The final "Summary of files generated" block enumerated every artefact (4 in CA\, 6 in Client\, 4 in Verifier\), matching the set produced by the manual workflow in Sections 2–4.3. This end-to-end automation demonstrates that the PKI system can be deployed programmatically in production environments, enabling integration into CI/CD pipelines or scheduled security workflows.

5. Generated Keys and Certificates

- 1- `ca_certificate.pem` — CA self-signed root certificate.



`ca_certificate.pem`

- 2- `bob_public.key` — Bob's RSA 2048-bit public key.



`bob_public.key`

- 3- `bob_csr.pem` — Bob's Certificate Signing Request.



`bob_csr.pem`

- 4- `bob_certificate.pem` — Bob's X.509 certificate issued by the CA.



`bob_certificate.pem`

- 5- `document.txt` — The original signed document.



`document.txt`

- 6- `signature.bin` — The SHA-256 digital signature.



`signature.bin`

- 7- `signature_sha512.bin` — The SHA-512 digital signature (optional feature).



signature_sha512.bin

8- bob_expired_cert.pem — Expired certificate for expiration simulation (optional feature).



bob_expired_cert.pem

9- revoked_certs.txt — Manual Certificate Revocation List (optional feature).



revoked_certs.txt

10- pki_automation.py — Python automation script (optional feature).



pki_automation.py

6. Conclusion

The mini-PKI system successfully demonstrates all core components of a Public Key Infrastructure: key generation, certificate signing requests, certificate issuance by a trusted CA, document signing, and signature verification. The system correctly handles both valid and invalid verification scenarios, proving document integrity, signer authentication, and non-repudiation.

All four optional features were implemented:

- (1) hash algorithm comparison showing SHA-256 vs SHA-512 behaviour, (
- 2) certificate expiration simulation demonstrating validity enforcement,
- (3) certificate revocation using a manual CRL to reject compromised certificates, and
- (4) Python automation scripting of the entire PKI workflow. These optional features further demonstrate the robustness, security controls, and real-world applicability of the PKI framework using NIST-approved algorithms.